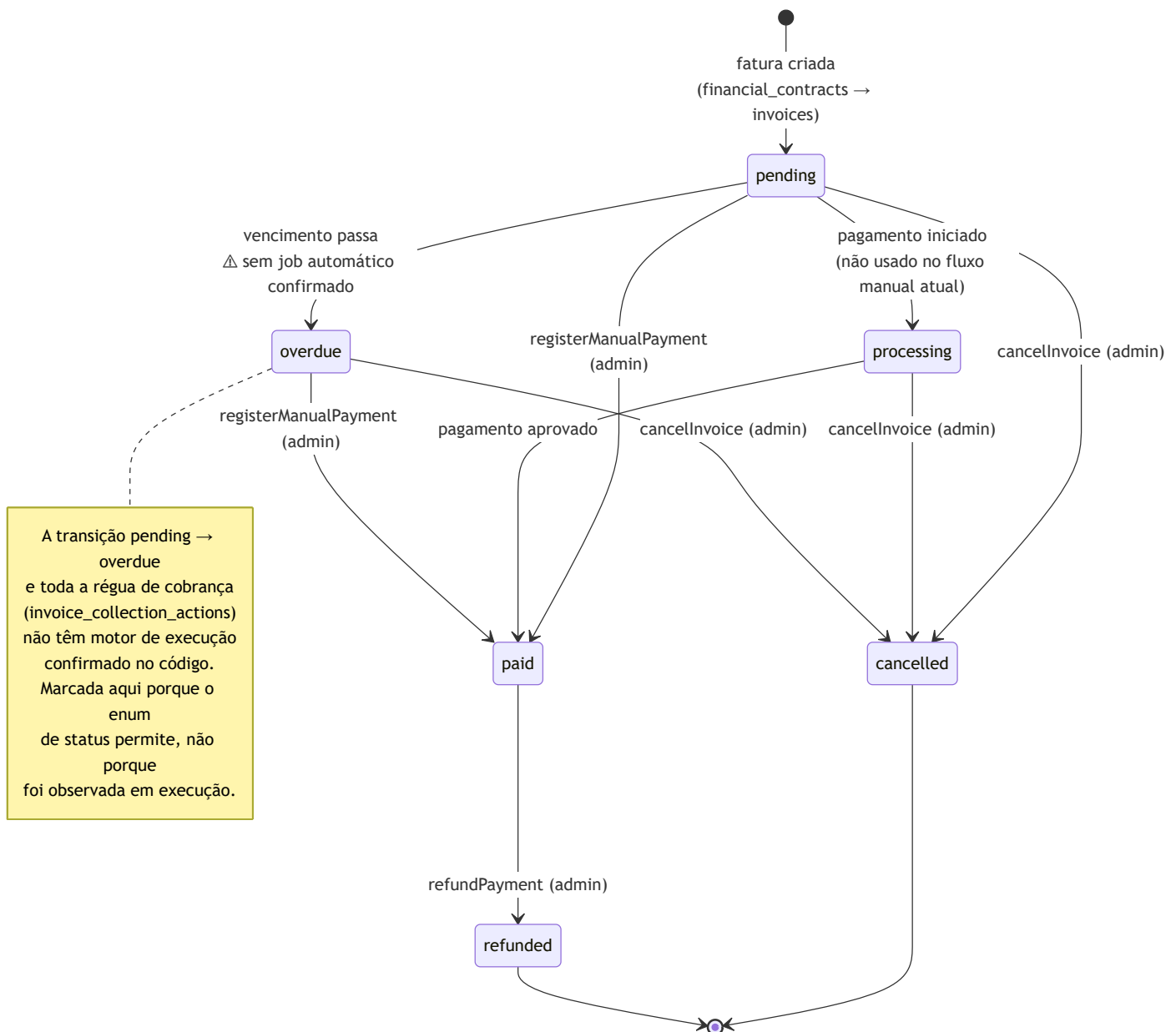


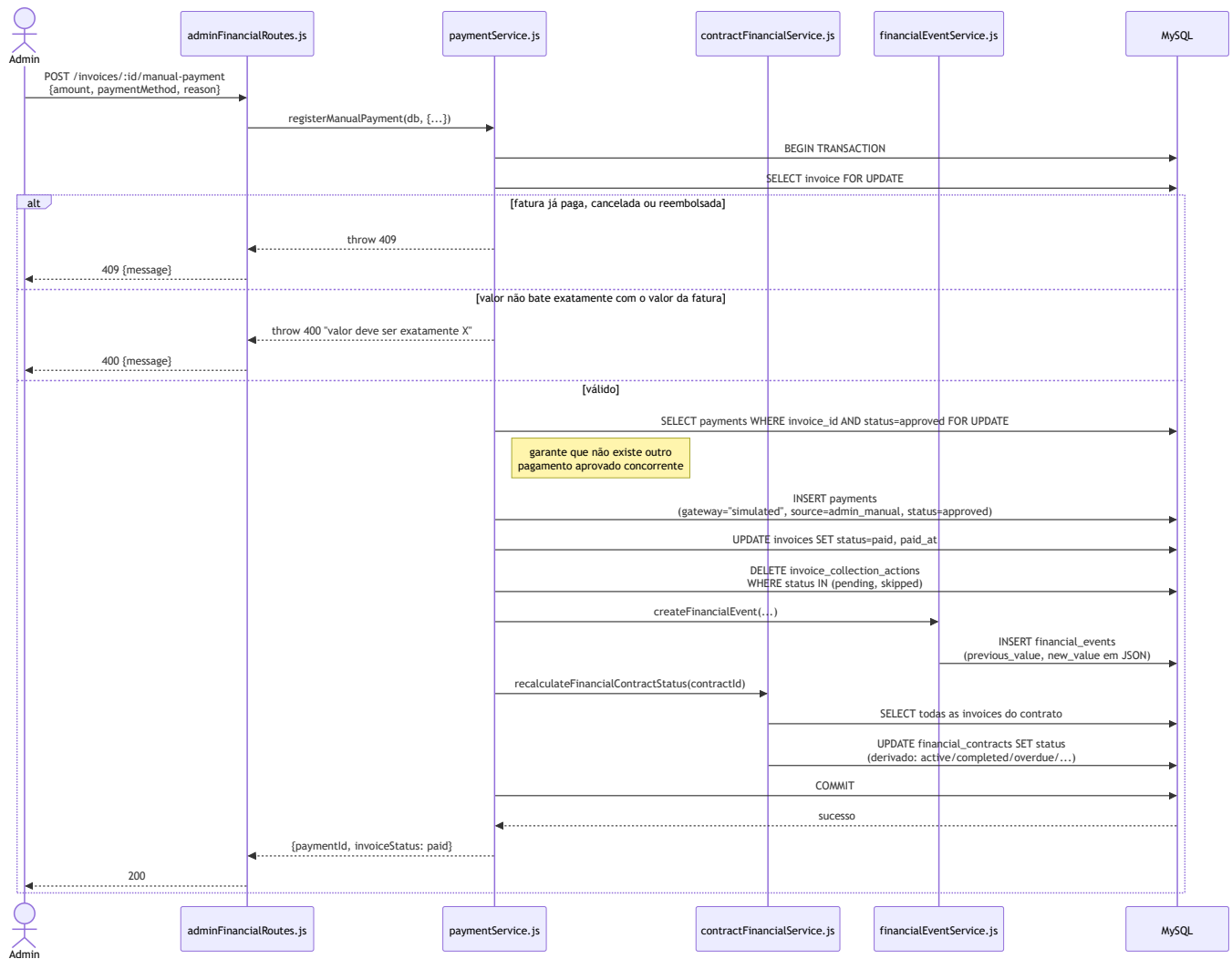
Diagrama — Fluxo Financeiro

Dois diagramas: o ciclo de vida de uma fatura (máquina de estados) e o fluxo de registro de pagamento manual pelo admin (sequência), o único mecanismo de pagamento com código de aplicação real hoje. Nomes reais: `paymentService.js` , `contractFinancialService.js` , `financialEventService.js` .

1. Máquina de estados de uma fatura (`invoices.status`)



2. Registro de pagamento manual (fluxo real e implementado)



Que problema este diagrama esclarece

O financeiro é o domínio com mais estados e mais regras condicionais do CourseHub. Sem um diagrama, é fácil presumir que existe um fluxo automatizado completo (cobrança → lembrete → atraso → bloqueio) só porque o schema (`invoice_collection_actions`) sugere isso. O primeiro diagrama existe especificamente para deixar visualmente claro **onde a implementação real termina e o desenho de schema planejado começa** — a nota anexada ao estado `overdue` é o elemento mais importante deste documento.

Como interpretar os elementos

- No diagrama de estados, toda transição sem anotação de "job automático" corresponde a uma chamada de API real, feita por um admin (`invoiceService.js` , `invoiceCancellationService.js` , `paymentRefundService.js` , `paymentService.js`) — não há transição de status de fatura que aconteça sozinha no sistema hoje, exceto a marcada com [△](#).
- No diagrama de sequência, os três `SELECT ... FOR UPDATE` (fatura, checagem de pagamento concorrente) são bloqueios de linha explícitos no código — existem para impedir que dois registros de pagamento simultâneos aprovelem a mesma fatura duas vezes.
- `financial_contracts.status` nunca aparece como algo que o admin define diretamente em nenhum dos dois diagramas — ele é sempre a **saída** de `recalculateFinancialContractStatus` , nunca uma **entrada**.

Que decisões arquiteturais este diagrama evidencia

- **Sem suporte a pagamento parcial:** o diagrama de sequência mostra explicitamente a rejeição quando o valor não bate exatamente com o valor da fatura — reflete a regra de negócio documentada em `business-rules.md` , e não é um detalhe de validação incidental, é uma decisão central do modelo de cobrança.
- **Toda mudança financeira gera um evento de auditoria antes do commit** — `createFinancialEvent` acontece dentro da mesma transação que a mudança de estado, nunca depois de forma assíncrona. Se a auditoria falhar, a mudança inteira é revertida.
- **Cancelamento de fatura limpa a régua de cobrança pendente** (`DELETE invoice_collection_actions WHERE status IN (pending, skipped)`) — mesmo sem motor de execução, o código já trata a tabela como algo que precisa ser mantido consistente, evidência de que a régua foi desenhada para ser ativada no futuro sem exigir uma limpeza retroativa de dados órfãos.

Trade-offs que este fluxo representa

- **gateway: "simulated" hardcoded em todo pagamento manual:** em vez de rotear por `services/paymentGateway/simulatedGateway.js` (que existe mas está desconectado), o código grava a string diretamente. Simplicidade imediata em troca de um módulo de gateway que hoje não tem nenhum consumidor real — um sinal de que, se um gateway de pagamento real for integrado no futuro, esse ponto do código precisa ser revisitado, não só estendido.
- **Recalcular o status do contrato lendo todas as suas faturas a cada mudança:** correto e simples, mas é uma leitura $O(\text{faturas do contrato})$ toda vez que qualquer fatura muda — aceitável

para o volume esperado (um contrato tem tipicamente até algumas dezenas de parcelas), não otimizado para portfólios de cobrança muito maiores.

- **Régua de cobrança modelada mas não executada:** permite que o produto avance em outras áreas sem bloquear no desenho da automação de cobrança, mas cria risco de alguém (erroneamente) presumir, ao ver a tabela `invoice_collection_actions` populável, que atrasos já disparam ações automáticas hoje.

Como este diagrama deve evoluir

Quando a régua de cobrança automatizada for implementada, o diagrama de estados perde a anotação de aviso na transição `pending` → `overdue` e ganha um terceiro diagrama de sequência mostrando o job/rotina que a executa. Se um gateway de pagamento real (Pix/boleto/cartão via provedor externo) for integrado, o diagrama de sequência precisa de um novo bloco `processing` → `paid` via webhook do provedor, hoje ausente porque o estado `processing` não é alcançado por nenhum código atual.